

Genesis PRD vs. Current Codebase Gap Analysis

This document details the exact architectural, database, and interface gaps between the current codebase implementation and the new Genesis PRD v1.0 specifications.

1. Onboarding & Discovery Flow

Step	Current Implementation	Genesis PRD Specification	Action Required
Start	Direct route to simple questionnaire.	SCREEN_01 (Splash) & SCREEN_02 (Future Self Hook) : Animated entrance with future self potential slider.	Build Splash and Future Self slider.
Details	Basic signup route.	SCREEN_03 (Account Creation) : Basic student profile inputs.	Map signup inputs to Genesis schema.
Assessment	Simple 5-question career target form.	SCREEN_04 (Identity Discovery) : 20-30 personality/interest questions.	Design 30-question interactive assessment module.
Simulations	None.	SCREEN_05 (Scenario Simulations) : 5-10 interactive situational judgment cards.	Build interactive scenario selector component.
Speech	None.	SCREEN_06 (Communication Assessment) : Voice input audio recording/speech capture.	Integrate browser Web Speech/Audio API capture.
Reveal	Redirects to dashboard.	SCREEN_07 (Blueprint Reveal) : Trait analysis (Primary, Secondary, Hidden) & Potential Score page.	Design trait visualization page with blueprint traits.

2. Home Dashboard & Telemetry HUDs

Metric/Panel	Current Implementation	Genesis PRD Specification	Action Required
Main HUD	Radial ATS Career Score, segmented Trust Score, and XP level.	Header Wireframe: Avatar, Current Level (Human Level formula), Potential Score, and Current Title.	Re-arrange header layout; implement Human Level Formula .
Future Self	Standard card recommending next step.	Future Self Progress Section: Clickable timeline for Current Self → Future Self → Maximum Self.	Build 3-stage timeline progress widget.
Quest Lists	Standard recommended missions list.	Quests Section: Separated into "Today's Main Quest" (large card) and "Side Quests" (smaller cards).	Reformat quest layout to match.
Growth Log	Standard activity logs feed.	Recent Growth Section: Dynamic logs/timeline showing quest completions and XP.	Align feed filter to display evolution logs.

3. Quest Hub & Details

Feature	Current Implementation	Genesis PRD Specification	Action Required
Structure	Vertical modules list; handles coding quests and classes.	Quest Hub Wireframe: Top progress counters, tabs for Recommended and 6 categories (Communication, Leadership, Career, Execution, Research, Simulation).	Re-layout quest index with skill tabs.
Quest Card	Compact card with title and link.	Quest Card Wireframe: Title, Difficulty, XP Reward, Time Estimate, and Start button.	Update card properties to render difficulty/time.
Submission	Editor compiler (runs local tests).	Quest Details Screen: Objective lists, plus support for 5 submission formats : Text, Voice, Video, Document, and Project.	Extend compiler screen to support audio/video/file uploads.

4. Evolution Dashboard & Career DNA

Feature	Current Implementation	Genesis PRD Specification	Action Required
Dimensions	9 skills (ATS score, communication, coding, resume completeness, etc.).	10 core dimensions: Communication, Leadership, Execution, Problem Solving, Creativity, Adaptability, Consistency, Collaboration, Learning Velocity, Strategic Thinking.	Re-map radar chart axes to PRD dimensions.
Analytics	Static skill list.	Evolution Dashboard: Trend Graphs, Historical Progress logs, and Growth Heatmaps.	Integrate chart widgets for heatmaps & history.

5. Digital Twin Screen

Feature	Current Implementation	Genesis PRD Specification	Action Required
Twin Interface	Chat box utilizing simple API response.	Digital Twin Screen: Chat UI, Memory Layer (Short-Term: 7 days, Long-Term: Vector), Decision Simulator, Career Guidance, Growth Analysis.	Build vector retrieval context pipeline.

6. Database Schema (Supabase)

Table	Current Fields	Genesis PRD Fields	Action Required
user_profiles	displayName, avatarUrl, ats_score, trust_score.	college, degree, graduation_year, lifecycle_state.	Add college/degree/state columns.
human_graph	None (derived).	10 dimension scores (communication, leadership, etc.), potential_score, level.	Create table human_graph linked to user_id.
blueprints	None.	primary_trait, secondary_trait, hidden_trait, potential_score.	Create table blueprints linked to user_id.
submissions	None.	quest_id, submission_type, content_url, text_content.	Create table submissions.
evaluations	None.	submission_id, dimension scores, feedback, raw response, confidence.	Create table evaluations.
twin_memory	None.	memory_type, memory_text, embedding_id.	Create table twin_memory with pgvector.

7. AI Orchestration Architecture

The client specifies a microservices gateway architecture where the client app routes requests through an API Gateway to Core Services. A dedicated **AI Orchestrator Layer** routes calls via a Provider Router (supporting failovers, retry policies, and cost-optimized switching between OpenAI, Claude, and Gemini) and returns strictly validated JSON schemas.

CLIENT (Mobile App / Web App)



API GATEWAY



CORE SERVICES (Auth, Profile, Quest, Evaluation, Graph, Twin)



AI ORCHESTRATOR LAYER



PROVIDER ROUTER (Failover logic: OpenAI -> Claude -> Gemini)



LLM PROVIDERS